

Lecture-13 [08 September 2026]

Value Function Approximation

Idea: Approximate the value function ($v_{\omega, \eta}^*$ or $v_{\omega, \eta}^\phi$ for some ϕ) using $I+1$ features $b_0, \dots, b_I$, where $I \ll |\mathcal{S}|$.

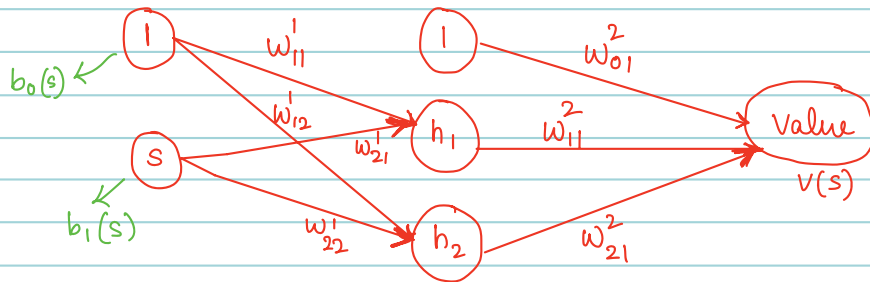
Linear architectures: $v(s) = \beta_0 b_0(s) + \dots + \beta_I b_I(s)$, $s \in \mathcal{S}$.

$$\underline{v} = B \underline{\beta}, \quad B = \begin{bmatrix} b_0(s_1) & \dots & b_I(s_1) \\ \vdots & & \vdots \\ b_0(s_m) & \dots & b_I(s_m) \end{bmatrix}_{m \times (I+1)}, \quad \underline{\beta} = \begin{bmatrix} \beta_0 \\ \vdots \\ \beta_I \end{bmatrix}.$$

Non-linear architectures: $v(s) \approx f_{\underline{\beta}}(b_0(s), \dots, b_I(s))$

Eg: $v(s) = \beta_0 + \beta_1 e^{\beta_2 s}$ non-linear function
[$b_0(s) \equiv 1$, $b_1(s) = s$]

Neural network-based architectures



Assuming the sigmoid activation function $\sigma(x) = \frac{1}{1 + e^{-x}}$,

we have:

$$\text{Input to } h_1: w_{11}^1 + w_{21}^1 s,$$

$$\text{Input to } h_2: w_{12}^1 + w_{22}^1 s.$$

$$v(s) = w_{01}^2 + w_{11}^2 \sigma(w_{11}^1 + w_{21}^1 s) + w_{21}^2 \sigma(w_{12}^1 + w_{22}^1 s)$$

$$= w_{01}^2 + w_{11}^2 \frac{1}{1 + e^{-(w_{11}^1 + w_{21}^1 s)}} + w_{21}^2 \frac{1}{1 + e^{-(w_{12}^1 + w_{22}^1 s)}}$$

The features $b_0, \dots, b_I$ are usually chosen so that B has full column rank, in which case $B^T B$ is invertible.

Parameter Estimation for Value Function Approximation: Linear Architectures

We saw that a linear architecture approximates value as

$$v(s) \approx \beta_0 + \beta_1 b_1(s) + \dots + \beta_I b_I(s), \quad s \in \mathcal{S}.$$

How do we determine $\beta_0, \dots, \beta_I$ that best approximate $v(\cdot)$?

Approximating $v_{\omega, \lambda}^\phi$ for a given $\phi: \mathcal{S} \rightarrow \mathcal{A}$

Suppose that we wish to approximate $v_{\omega, \lambda}^\phi$ for a fixed $\phi: \mathcal{S} \rightarrow \mathcal{A}$.

Why approximation

We know that

$$v_{\omega, \lambda}^\phi = (\mathbf{I} - \lambda P_\phi)^{-1} r_\phi, \quad v_{\omega, \lambda}^\phi = L_\phi v_{\omega, \lambda}^\phi.$$

Although $v_{\omega, \lambda}^\phi$ has a closed-form solution, evaluating it is computationally expensive

$\rightarrow O(|\mathcal{S}|^3)$ operations; not feasible when $\mathcal{S}$ is large

Suppose now that we wish to approximate $v_{\omega, \lambda}^\phi$ using a linear architecture of the form $B\beta_-$.

How do we determine β_- ?

Case 1 [Only for understanding, not used in practice]: $v_{\omega, \lambda}^\phi$ already available.

Suppose $v_{\omega, \lambda}^\phi$ is already given to us. Then, there is of course no need to approximate it. Nonetheless, we ask the following question.

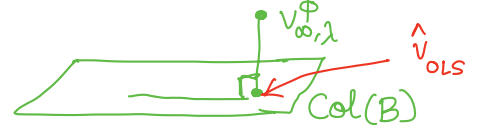
What β_- can we choose so that $B\beta_-$ is close to $v_{\omega, \lambda}^\phi$?

Let

$$\hat{\beta} \in \arg \min_{\beta \in \mathbb{R}^{I+1}} \|B\beta - v_{\omega, \lambda}^\phi\|_2 \iff \hat{\beta} \in \arg \min_{\beta \in \mathbb{R}^{I+1}} \|B\beta - v_{\omega, \lambda}^\phi\|_2^2.$$

The optimal choice $\hat{\beta}$ is the ordinary least squares solution given by

$$\hat{\beta}_{\text{OLS}} = \underbrace{(B^T B)^{-1} B^T}_{=: \Gamma} v_{\omega, \lambda}^\phi = \Gamma v_{\omega, \lambda}^\phi = \Gamma (\mathbf{I} - \lambda P_\phi)^{-1} r_\phi.$$



Furthermore, the best approximation is given by

$$\hat{v}_{ols}^{\phi} = B \hat{\beta}_{ols} = B (B^T B)^{-1} B^T v_{w,\lambda}^{\phi} = \Pi v_{w,\lambda}^{\phi} = \Pi (I - \lambda P_{\phi})^{-1} x_{\phi}.$$

↓ belongs to Col(B) $=: \Pi$

Important matrices:

$$\Gamma = (B^T B)^{-1} B^T,$$

$$\Pi = B (B^T B)^{-1} B^T = B \Gamma.$$

→ projection matrix; $\Pi^2 = \Pi$, $\Pi^T = \Pi$.

Case 2: $v_{w,\lambda}^{\phi}$ not available

Suppose now that we wish to approximate $v_{w,\lambda}^{\phi}$ using a linear form $B\beta$.

Because $v_{w,\lambda}^{\phi}$ satisfies

$$v_{w,\lambda}^{\phi} = L_{\phi} v_{w,\lambda}^{\phi},$$

$$L_{\phi} v = x_{\phi} + \lambda P_{\phi} v$$

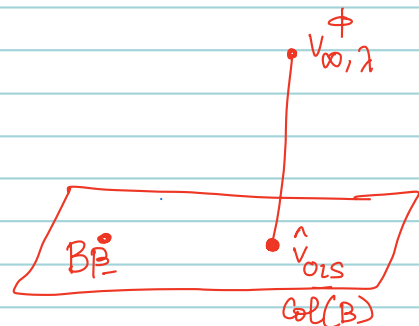
one possibility is to choose β such that $B\beta$ is close to $L_{\phi} B\beta$. This technique is called Bellman residual minimisation.

Bellman residual minimisation:

$$\hat{\beta}_{-BR} \in \operatorname{argmin}_{\beta \in \mathbb{R}^{I+1}} \|B\beta - L_{\phi} B\beta\|_2^2.$$

Another possibility is as follows: observe that

$$\hat{\beta}_{ols} \in \operatorname{argmin}_{\beta \in \mathbb{R}^{I+1}} \|B\beta - v_{w,\lambda}^{\phi}\|_2^2$$



The $\hat{\beta}_{ols}$ that minimises 2-norm blw $B\beta$ and $v_{w,\lambda}^{\phi}$ is also the one that minimises the 2-norm blw $B\beta$ and $\Pi v_{w,\lambda}^{\phi}$.

$$\hat{\beta}_{ols} \in \operatorname{argmin}_{\beta \in \mathbb{R}^{I+1}} \|B\beta - \Pi v_{w,\lambda}^{\phi}\|_2^2$$

$$= \operatorname{argmin}_{\beta \in \mathbb{R}^{I+1}} \|B\beta - \Pi L_{\phi} v_{w,\lambda}^{\phi}\|_2^2,$$

$$\Pi = B (B^T B)^{-1} B^T.$$

One may consider replacing $v_{w,\lambda}^{\phi}$ with $B\beta$ to get

$$\hat{\beta} \in \operatorname{argmin}_{\beta \in \mathbb{R}^{I+1}} \|B\beta - \Pi L_{\phi} B\beta\|_2^2.$$

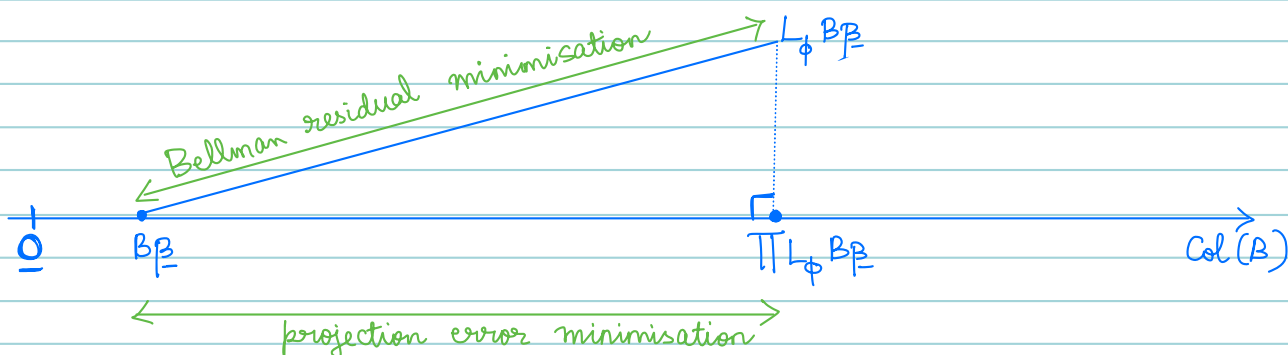
This technique is called projection error minimisation.

Projection error minimisation:

$$\hat{\beta}_{PE} \in \underset{\beta \in \mathbb{R}^{I+1}}{\operatorname{argmin}} \|B\beta - \Pi L_{\phi} B\beta\|_2^2, \quad \Pi = B(B^T B)^{-1} B^T.$$

Remark:

Given β , $L_{\phi} B\beta$ may not belong to $\operatorname{Col}(B)$, but $\Pi L_{\phi} B\beta \in \operatorname{Col}(B)$.



Example [to demonstrate difference b/w BR and PE minimisation techniques]

Let $\mathcal{S} = \{1, 2\}$, and let $\phi: \mathcal{S} \rightarrow \mathbb{A}$ be such that

$$r_{\phi} = \begin{bmatrix} 2 \\ 8 \end{bmatrix}, \quad P_{\phi} = \begin{bmatrix} 1/4 & 3/4 \\ 1/10 & 9/10 \end{bmatrix}.$$

$$P_{\phi}(s'|s) = P(s'|s, \phi(s))$$

Let $b(s) = s$ be the only basis function. Then,

$$B = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad B\beta = \begin{bmatrix} \beta \\ 2\beta \end{bmatrix}, \quad \beta \in \mathbb{R}.$$

The Bellman residual error is given by

$$\begin{aligned} \|B\beta - L_{\phi} B\beta\|_2 &= \|B\beta - (r_{\phi} + \lambda P_{\phi} B\beta)\|_2 \\ &= \left\| \begin{bmatrix} \beta \\ 2\beta \end{bmatrix} - \begin{bmatrix} 2 \\ 8 \end{bmatrix} - \begin{bmatrix} 1.75\lambda\beta \\ 1.9\lambda\beta \end{bmatrix} \right\|_2 \\ &= \left\| \begin{bmatrix} (1-1.75\lambda)\beta - 2 \\ (2-1.9\lambda)\beta - 8 \end{bmatrix} \right\|_2. \end{aligned}$$

On the other hand,

$$\Pi = B(B^T B)^{-1} B^T = \frac{1}{5} \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix},$$

and the projection error is given by

$$\|B\beta - \Pi_{L_\phi} B\beta\|_2 = \left\| \begin{bmatrix} (1 - \frac{10}{9}\lambda)\beta - 2.4 \\ (2 - \frac{20}{9}\lambda)\beta - 4.8 \end{bmatrix} \right\|_2.$$

If $\lambda = 0.6$, then

$$V_{\infty, \lambda}^\phi = (I - \lambda P_\phi)^{-1} r_\phi = \begin{bmatrix} 18.4 \\ 25.1 \end{bmatrix}, \quad \Pi V_{\infty, \lambda}^\phi = \begin{bmatrix} 10.09 \\ 20.18 \end{bmatrix},$$

$$B\hat{\beta}_{BR} = \begin{bmatrix} 9.14 \\ 18.27 \end{bmatrix}, \quad B\hat{\beta}_{PE} = \begin{bmatrix} 10.78 \\ 21.55 \end{bmatrix}.$$

The Least Squares Policy Evaluation (LSPE) Algorithm

Based on PE minimisation idea, the LSPE algorithm is designed as follows.

– Initialise: Specify $\beta \in \mathbb{R}^{I+1}$, $\epsilon > 0$, $\Delta > \epsilon$.

– Iterate: While $\Delta \geq \epsilon$:

$$\bullet \underline{V}' \leftarrow B\beta$$

$$\bullet \underline{V} \leftarrow L_\phi \underline{V}' = r_\phi + \lambda P_\phi \underline{V}'$$

$$\bullet \beta' \leftarrow \Gamma \underline{V}$$

$$\beta' \leftarrow \underset{\underline{x}}{\operatorname{argmin}} \|\underline{V} - B\underline{x}\|_2^2$$

$$\bullet \Delta \leftarrow \|\beta' - \beta\|_2$$

$$\bullet \beta \leftarrow \beta'.$$

– Terminate: Return β .

Let $\hat{\beta}_{LSPE}$ denote the β vector returned by the LSPE algorithm above.

Notice that any two successive iterates β, β' of LSPE are related as

$$\beta' = \Gamma(r_\phi + \lambda P_\phi B\beta) = \underbrace{\Gamma r_\phi + \lambda \Gamma P_\phi B\beta}_{g(\beta)}.$$

If the matrix $\lambda \Gamma P_\phi B$ is a contraction, then $g: \mathbb{R}^{I+1} \rightarrow \mathbb{R}^{I+1}$ is a contraction, as a result of which the sequence $\{\beta_k\}_{k \geq 0}$ ← arising from a non-stopping version of LSPE converges to $\hat{\beta}^*$ satisfying

$$g(\underline{\beta}^*) = \underline{\beta}^* \Leftrightarrow \Gamma \underline{x}_\phi + \lambda \Gamma P_\phi B \hat{\underline{\beta}}^* = \hat{\underline{\beta}}^*$$

$$\Leftrightarrow (\mathbf{I} - \lambda \Gamma P_\phi B) \hat{\underline{\beta}}^* = \Gamma \underline{x}_\phi, \quad \Gamma = (B^T B)^{-1} B^T$$

If $\lambda \Gamma P_\phi B$ is indeed a contraction, then $(\mathbf{I} - \lambda \Gamma P_\phi B)^{-1}$ exists, and

$$\hat{\underline{\beta}}^* = (\mathbf{I} - \lambda \Gamma P_\phi B)^{-1} \Gamma \underline{x}_\phi.$$

Example [to demonstrate that $(\mathbf{I} - \lambda \Gamma P_\phi B)$ may not be invertible]

$$\text{Let } S = \{s_1, s_2\}, \quad \lambda = \frac{5}{5.4}, \quad P_\phi = \begin{bmatrix} 0.2 & 0.8 \\ 0.2 & 0.8 \end{bmatrix}, \quad B = \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

Then,

$$\Gamma = (B^T B)^{-1} B^T = \frac{1}{5} \begin{bmatrix} 1 & 2 \end{bmatrix}, \quad \Gamma P_\phi B = \frac{5.4}{5}$$

and therefore

$$\begin{aligned} \lambda \Gamma P_\phi B &= \frac{5}{5.4} \cdot \frac{1}{5} \begin{bmatrix} 1 & 2 \end{bmatrix} \begin{bmatrix} 0.2 & 0.8 \\ 0.2 & 0.8 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} \\ &= 1. \end{aligned}$$

Thus, $\mathbf{I} - \lambda \Gamma P_\phi B$ (in this a 1×1 matrix) is not invertible!

Parameter Estimation for Value Function Approximation: Non-Linear Architectures

We will now modify the LSPE algorithm by replacing

$$v' \leftarrow B \underline{\beta}$$

with the line

$$v'(s) \leftarrow \underbrace{f_{\underline{\beta}}(b_0(s), \dots, b_I(s))}_{=: f_{\underline{\beta}}(s)} \quad \forall s \in \mathcal{S}.$$

$$\text{Let } \underline{f}_{\underline{\beta}} := \begin{bmatrix} f_{\underline{\beta}}(s_1) \\ \vdots \\ f_{\underline{\beta}}(s_m) \end{bmatrix}.$$

Algorithm 9.2 [Least Squares Policy Evaluation [LSPE] for non-linear architectures]

(In non-linear architectures, $\underline{\beta}$ need not be of length $I+1$)

- Initialise : Specify $\underline{\beta}$, $\epsilon > 0$, $\Delta > \epsilon$.

- Iterate: While $\Delta \geq \epsilon$:

- $\underline{v}' \leftarrow \underline{f}_{\underline{\beta}}$ $v'(s) = f_{\underline{\beta}}(s) = f_{\underline{\beta}}(b_0(s), \dots, b_I(s))$
- $\underline{v} \leftarrow L_{\phi} \underline{v}' = \alpha_{\phi} + \lambda P_{\phi} \underline{v}'$
- $\underline{\beta}' \in \arg \min_{\underline{\beta}} \|\underline{f}_{\underline{\beta}} - \underline{v}\|_2^2$
- $\Delta \leftarrow \|\underline{\beta}' - \underline{\beta}\|_2$
- $\underline{\beta} \leftarrow \underline{\beta}'$

- Terminate: Return $\underline{\beta}$.

Remarks on LSPE (linear/non-linear architectures)

① Computation of $\underline{v} \leftarrow L_{\phi} \underline{B} \underline{\beta}$, $\underline{\beta}' \leftarrow \Gamma \underline{v}$ in linear case, and

$\underline{v} \leftarrow L_{\phi} \underline{f}_{\underline{\beta}}$, $\underline{\beta}' \in \arg \min_{\underline{\beta}} \|\underline{f}_{\underline{\beta}} - \underline{v}\|_2$ in non-linear case

can be done only for a subset of states $\mathcal{S}' \subset \mathcal{S}$ if $\mathcal{S}$ is large.

For this, suppose that $\mathcal{S}' = \{s'_1, \dots, s'_n\}$. Let

$$v(s') \leftarrow r(s', \phi(s')) + \sum_{s'' \in \mathcal{S}} P(s'' | s', \phi(s')) \cdot \left(\sum_{i=0}^I \beta_i b_i(s'') \right), \quad s' \in \mathcal{S}'$$

$$\text{Set } \underline{v} = \begin{bmatrix} v(s'_1) \\ \vdots \\ v(s'_n) \end{bmatrix}, \quad \underline{\bar{B}} = \begin{bmatrix} b_0(s'_1) & \dots & b_I(s'_1) \\ \vdots & & \vdots \\ b_0(s'_n) & \dots & b_I(s'_n) \end{bmatrix}_{n \times (I+1)}$$

$$\text{Set } \underline{\beta}' = \Gamma \underline{v} = (\underline{\bar{B}}^T \underline{\bar{B}})^{-1} \underline{\bar{B}}^T \underline{v} = \arg \min_{\underline{\beta}} \|\underline{\bar{B}} \underline{\beta} - \underline{v}\|_2^2.$$

Similar extensions in the non-linear setting hold.

② Under some nice regularity conditions, the LSPE algorithm converges. For instance, if P_{ϕ} is ergodic with unique stationary distribution ξ_{ϕ} satisfying $\xi_{\phi} P_{\phi} = \xi_{\phi}$, and B has full column rank, then

the iterates of LSPE converge under the norm $\|\cdot\|_w$ given by

$$\|\underline{x}\|_w = \sqrt{\underline{x}^T \underline{W} \underline{x}}, \quad \underline{W} = \text{diag}(\xi_{\phi}(s_1), \dots, \xi_{\phi}(s_m)).$$

Because all norms on $\mathbb{R}^{|S|}$ are equivalent, this implies convergence in $\|\cdot\|_2$.

will be presented
in the next
lecture

③ In a neural network-based implementation of LSPE, a neural network must be trained to obtain $\underline{\beta}'$ such that

$$\underline{\beta}' \in \underset{\underline{\theta}}{\operatorname{argmin}} \frac{1}{2M} \sum_{i=1}^M [f_{\underline{\theta}}(s_i) - v(s_i)],$$

Mahalanobis norm:

For any symmetric PSD A ,

$$\|\underline{x}\|_A := \sqrt{\underline{x}^T A \underline{x}}$$

where $f_{\underline{\theta}}$ represents the NN's i/p-o/p relationship with weights $\underline{\theta}$,

and $\underline{v} = [v(s_i) : i=1, \dots, M]$ is from the step before $\underline{\beta}'$ computation in LSPE.
